

# ZEB

Lightweight Machine Identity Provider for  
Secure Machine-to-Machine Authentication

*Project Concept Note & Technical Reference*

Version 1.0 | Software Engineering Internship / University Project

# 1. Project Title

---

ZEB

Lightweight Machine Identity Provider for Secure Machine-to-Machine Authentication.

ZEB is an educational, from-scratch implementation of the core concepts behind enterprise Machine Identity and Access Management (IAM) platforms such as HashiCorp Vault, SPIFFE/SPIRE, and Keycloak. It is not a clone of any existing product — it is a learning-first system built to demonstrate, in working code, how machines authenticate and authorize each other in modern distributed systems.

## 2. Executive Summary

---

Modern applications are no longer single programs — they are made of many small services (microservices) that constantly talk to each other over the network. A checkout page might call an Order Service, which calls an Inventory Service, which calls a Payment Service. None of these are human users; they are machines calling other machines.

Today, many of these connections are secured with weak methods: hard-coded API keys, shared passwords, or no authentication at all. If one service is compromised, an attacker can often move freely to the next one.

ZEB solves this by giving every machine (service, script, or application) its own verifiable digital identity — similar to how a human employee has a username and password, but designed for machines. Each machine registers with ZEB, receives credentials, and must authenticate before it is allowed to call another service. ZEB issues short-lived, cryptographically signed tokens (JWTs) that prove who the machine is and what it is allowed to do, and it records every authentication event for auditing.

By building ZEB from scratch, the team will learn — hands-on — how real IAM systems generate credentials, issue tokens, validate signatures, enforce permissions, and log security events, without depending on any external identity platform.

## 3. Background

---

### 3.1 What is Identity and Access Management (IAM)?

IAM is the discipline of ensuring that the right entity (a person, service, or device) can access the right resource, at the right time, for the right reason. An IAM system typically answers three questions: Who are you? (authentication), What are you allowed to do? (authorization), and What did you do? (audit).

### 3.2 Human Identity vs. Non-Human (Machine) Identity

| Aspect            | Human Identity                       | Machine Identity                                      |
|-------------------|--------------------------------------|-------------------------------------------------------|
| Who it represents | A person (employee, customer)        | A service, script, container, or device               |
| Login method      | Username + password, biometrics, SSO | Client ID + client secret, certificates, tokens       |
| Session length    | Minutes to hours, often interactive  | Very short-lived, fully automated, no human present   |
| Volume            | Thousands of users in a large org    | Often 10-100x more machine identities than human ones |
| Example           | An engineer logging into a dashboard | The Order Service calling the Payment Service         |

### 3.3 Why Machine Identity Has Become Important

As organizations moved from single large applications to distributed architectures, the number of machine-to-machine connections exploded. Machines now vastly outnumber human users in most systems, yet they historically received far less security attention.

- Microservices: a single request may pass through 5-20 internal services, each needing to prove its identity to the next.
- APIs: internal and third-party APIs need a reliable way to know which client is calling.
- Cloud computing: workloads are created and destroyed dynamically, so identities must be issued and revoked automatically, not manually.
- Kubernetes and containers: pods are ephemeral and need identity that is provisioned at startup, not hard-coded into images.
- AI agents and automation: autonomous scripts and AI agents now call internal systems on their own, without a human clicking anything, making strong machine identity essential.
- Distributed systems: with no single perimeter, every internal call must be verified — this is the foundation of a Zero Trust approach.

### 3.4 Current Challenges Without Machine Identity Management

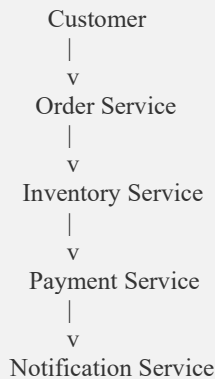
- Long-lived, hard-coded secrets stored directly in source code or configuration files.
- The same API key reused across many services, so one leak compromises everything.
- No standard way to know which service actually made a request.
- No central place to disable a compromised service quickly.

- No audit trail of which machine accessed what, and when.

## 4. Problem Statement

---

Consider a simplified e-commerce checkout flow, similar in spirit to how large platforms such as Amazon are structured internally:



Each arrow in this diagram is a machine-to-machine call. The Order Service is not a person — it is a program that must prove to the Inventory Service that it really is the Order Service, and that it is allowed to reserve stock. Without a proper identity system, teams typically fall back on weak patterns:

### 4.1 API Keys

A static string is generated once and placed in a configuration file. It rarely expires, is often shared across environments, and grants access until someone remembers to rotate it.

### 4.2 Shared Secrets

Multiple services use the same secret to call a downstream API. If one service is breached, the attacker gains the same access as every other service that shares the secret.

### 4.3 Machine Impersonation

Because there is no verifiable identity, any process that knows the shared key or is placed inside the network can pretend to be the Order Service and call the Payment Service directly.

### 4.4 Unauthorized Service Access

Without fine-grained permissions, a service that only needs to read inventory levels may technically be able to call the Payment Service too, simply because both trust the same network zone.

### 4.5 Credential Leakage

Secrets committed to source control, logged by accident, or exposed in error messages remain valid for a long time because there is no central system tracking or rotating them.

### 4.6 Lack of Centralized Identity

There is no single source of truth listing which machines exist, which are active, and which have been disabled — making incident response slow and error-prone.

## 5. Proposed Solution

---

ZEB is a self-contained Machine Identity Provider that every internal service registers with and authenticates against before calling another service. It provides the following core capabilities:

- Machine Registration — every service is registered as a known machine identity with a unique ID.
- Client Credential Generation — ZEB issues a `client_id` and `client_secret` pair for each machine, similar to a username and password made for software.
- Authentication — a machine proves its identity by presenting valid credentials to ZEB.
- JWT Generation — after successful authentication, ZEB issues a signed, short-lived JSON Web Token (JWT) representing that machine's identity and permissions.
- JWT Validation — any service receiving a request can verify the token's signature and expiry without contacting ZEB directly for every call.
- Permission Management — administrators define what each machine is allowed to do (e.g., "read-inventory", "create-payment").
- Authorization — protected services check the permissions embedded in the token before performing an action.
- Audit Logging — every authentication attempt, token issuance, and access decision is recorded for traceability.

Together, these features let every service prove who it is, restrict what it is allowed to do, and leave a verifiable trail of activity — the three pillars of machine IAM.

## 6. Project Scope

---

### 6.1 Version 1 (MVP) — In Scope

- Register Machine
- List Machines
- Disable Machine
- Generate Client Credentials
- Authenticate Machine
- Generate JWT
- Validate JWT
- Assign Permissions
- Authorization Check
- Audit Logs
- Demo Applications

## 6.2 Out of Scope for Version 1

The following capabilities exist in real enterprise IAM products, but are intentionally excluded from Version 1 so the team can focus on getting the core authentication and authorization loop correct before adding complexity.

| Feature                          | Why it is excluded from V1                                                                                                              |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| RBAC (Role-Based Access Control) | V1 uses simple direct permission assignment; role hierarchies add design complexity better tackled once the permission model is proven. |
| Secret Rotation                  | Requires scheduling, versioning, and grace-period logic; not needed to demonstrate the core auth flow.                                  |
| Certificate-based Auth / mTLS    | Needs a certificate authority and TLS infrastructure; adds significant setup overhead for a learning MVP.                               |
| Kubernetes Integration           | Ties the project to a specific runtime; the core concepts should first work in a simple, portable environment.                          |
| Admin Dashboard (UI)             | V1 is API-first; a UI is a presentation layer that can be added once the backend is stable.                                             |
| Notifications                    | Alerting is an operational concern, not core to proving the identity model.                                                             |
| Reporting                        | Advanced analytics over audit data is a V2+ concern once enough log data exists.                                                        |
| Risk Assessment                  | Requires behavioral baselines and scoring models that depend on data the MVP has not yet collected.                                     |
| Multi-tenancy                    | Adds isolation and data-partitioning complexity unrelated to the core authentication mechanics.                                         |
| AI Agent Identity                | A specialized identity type best addressed after the general machine identity model is validated.                                       |

## 7. Functional Requirements

---

Each MVP capability is described below in terms of its purpose, inputs, outputs, workflow, an example request, and the expected result.

### 7.1 Register Machine

- Purpose: Create a new machine identity record so it can later obtain credentials.
- Inputs: machine name, description, owner/team.
- Outputs: a unique machine\_id and a status of "active".
- Workflow: Admin submits machine details -> ZEB validates uniqueness -> record stored -> machine\_id returned.

```
POST /machines
{
  "name": "order-service",
  "description": "Handles customer orders",
  "owner": "checkout-team"
}
```

Expected Result: HTTP 201 Created, with the new machine\_id in the response body.

## 7.2 List Machines

- Purpose: Let an administrator see all registered machines and their status.
- Inputs: optional filters (status, owner).
- Outputs: a list of machine records.
- Workflow: Admin calls the list endpoint -> ZEB queries the Machines table -> results returned.

```
GET /machines?status=active
```

Expected Result: HTTP 200 OK with a JSON array of machines.

## 7.3 Disable Machine

- Purpose: Immediately revoke a machine's ability to authenticate, e.g. if it is compromised.
- Inputs: machine\_id.
- Outputs: updated machine status of "disabled".
- Workflow: Admin disables machine -> status updated -> future authentication attempts for that machine are rejected.

```
PATCH /machines/{id}/disable
```

Expected Result: HTTP 200 OK; subsequent /oauth/token calls for this machine return 401 Unauthorized.

## 7.4 Generate Client Credentials

- Purpose: Issue the client\_id / client\_secret pair a machine uses to authenticate, similar to a login for software.
- Inputs: machine\_id.
- Outputs: client\_id (public) and client\_secret (shown once, stored as a hash).
- Workflow: Admin requests credentials for a machine -> ZEB generates a random secret -> secret hash stored -> plaintext secret returned once.

Expected Result: The plaintext secret is never stored or retrievable again — only the machine that received it should keep a copy.

## 7.5 Authenticate Machine

- Purpose: Verify a machine's credentials before issuing a token.
- Inputs: client\_id, client\_secret.
- Outputs: success or failure, leading to JWT issuance on success.
- Workflow: Machine submits credentials -> ZEB checks machine is active -> compares secret hash -> proceeds to JWT generation.

```
POST /oauth/token
{
  "client_id": "cid_8b21ac",
  "client_secret": "s3cr3t-value",
  "grant_type": "client_credentials"
}
```

Expected Result: HTTP 200 OK with an access token on success; HTTP 401 Unauthorized on invalid or disabled credentials.

## 7.6 Generate JWT

- Purpose: Issue a signed token representing the authenticated machine's identity and permissions.
- Inputs: machine\_id, assigned permissions.
- Outputs: a signed JWT string with an expiry time.
- Workflow: After successful authentication -> ZEB builds claims (sub, permissions, exp) -> signs the token with its private/secret key -> returns it.

```
{
  "sub": "order-service",
  "permissions": ["inventory:read", "payment:create"],
  "iat": 1753000000,
  "exp": 1753000900
}
```

Expected Result: A compact JWT string (header.payload.signature) valid for a short period, e.g. 15 minutes.

## 7.7 Validate JWT

- Purpose: Allow a protected service to confirm a token is genuine and unexpired.
- Inputs: a JWT presented in the Authorization header.
- Outputs: valid/invalid decision plus the decoded claims.
- Workflow: Receiving service extracts token -> verifies signature -> checks expiry -> reads permissions.

```
POST /oauth/verify
Authorization: Bearer <jwt>
```

Expected Result: HTTP 200 OK with decoded claims if valid; HTTP 401 Unauthorized if the signature is invalid or the token has expired.

## 7.8 Assign Permissions

- Purpose: Define what actions a given machine is allowed to perform.
- Inputs: machine\_id, list of permission strings.
- Outputs: updated permission set for that machine.
- Workflow: Admin assigns permissions -> stored in the Permissions table -> included in future JWTs issued to that machine.

Expected Result: The next token issued to the machine reflects the updated permission list.

## 7.9 Authorization Check

- Purpose: Ensure a machine can only perform actions its token explicitly allows.
- Inputs: decoded JWT permissions, the action being requested.
- Outputs: allow or deny decision.
- Workflow: Service validates JWT -> checks if the required permission is present in the token's permission list -> allows or rejects the call.

Expected Result: A request for an action not present in the token's permissions is rejected with HTTP 403 Forbidden.

## 7.10 Audit Logs

- Purpose: Provide a traceable record of authentication and authorization events.
- Inputs: event type, machine\_id, timestamp, outcome.
- Outputs: a queryable log of security-relevant events.
- Workflow: Every authentication attempt and authorization decision is written to the AuditLogs table.

```
GET /audit?machine_id=order-service
```

Expected Result: HTTP 200 OK with a chronological list of events for that machine.

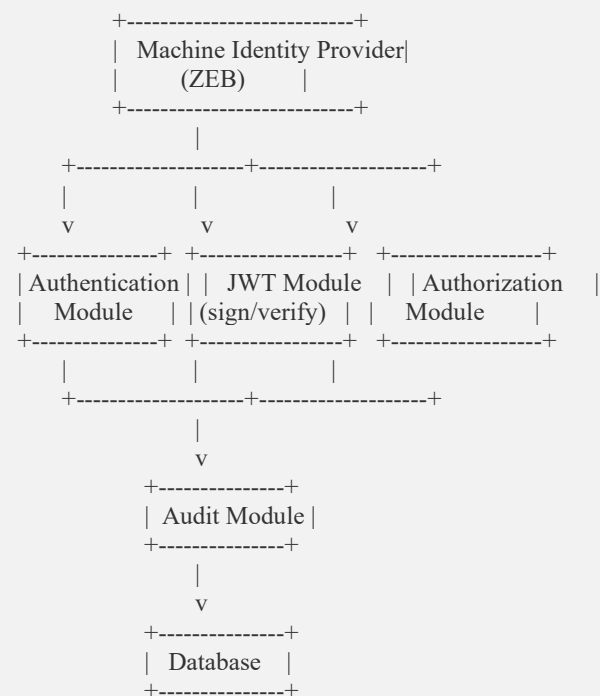
## 7.11 Demo Applications

A small set of sample services (Order, Inventory, Payment, Notification) will be built to demonstrate ZEB in a realistic, end-to-end scenario, described in Section 14.

# 8. Non-Functional Requirements

| Category        | Requirement                                                                                                                                      |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Security        | Secrets are stored only as salted hashes; JWTs are signed and short-lived; all traffic is assumed to run over HTTPS in any non-local deployment. |
| Performance     | Token validation should complete in well under 50ms so it does not become a bottleneck for service-to-service calls.                             |
| Availability    | The MVP targets single-instance reliability; graceful error responses are required even under partial failure (e.g., database unreachable).      |
| Scalability     | The service layer should be stateless so multiple instances can be added behind a load balancer in future versions.                              |
| Maintainability | Code is organized into clear modules (Auth, JWT, Authorization, Audit) with documented interfaces between them.                                  |
| Usability       | API responses use clear, consistent JSON error formats so demo services are easy to integrate against.                                           |
| Reliability     | Failed authentication attempts must never silently succeed; invalid or expired tokens are always rejected.                                       |

## 9. High-Level System Architecture

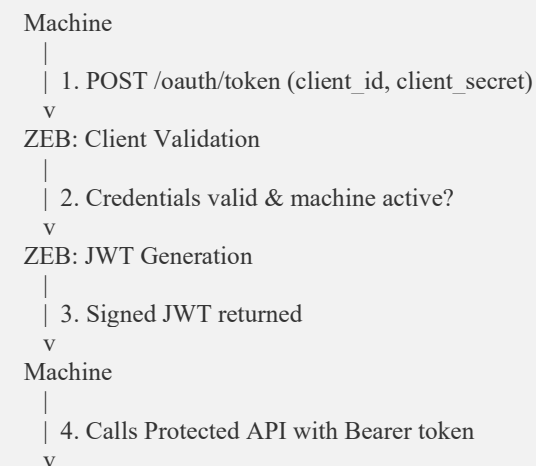


Component overview:

- Authentication Module: verifies `client_id/client_secret` pairs and checks machine status.
- JWT Module: signs new tokens and verifies incoming tokens' signatures and expiry.
- Authorization Module: compares a token's permissions against the action a service is trying to perform.
- Audit Module: writes a durable record of every authentication and authorization event.
- Database: persists Machines, Permissions, and AuditLogs (see Section 12).

Demo services (Order, Inventory, Payment) sit outside ZEB and act as its clients — each registers as a machine, authenticates to receive a token, and calls other demo services using that token.

## 10. Authentication Flow



Protected Service: JWT Validation

|

| 5. Signature + expiry OK?

v

Protected Service: Permission Check

|

| 6. Permission present?

v

Access Granted

Step-by-step explanation:

- 1. The calling machine sends its `client_id` and `client_secret` to ZEB's token endpoint.
- 2. ZEB checks that the machine exists, is active (not disabled), and that the secret hash matches.
- 3. On success, ZEB builds a JWT containing the machine's identity and permissions, signs it, and returns it with a short expiry.
- 4. The machine attaches this JWT as a Bearer token when calling another service.
- 5. The receiving service validates the token's signature and confirms it has not expired.
- 6. The receiving service checks whether the required permission is present in the token before allowing the action, and the event is recorded in the audit log.

## 11. Authorization Flow

---

Permissions are stored centrally in ZEB's Permissions table as simple strings tied to a `machine_id`, for example `inventory:read` or `payment:create`. This keeps the model easy to reason about for an MVP, while still being expressive enough for realistic demo scenarios.

When a token is issued, ZEB looks up all permissions currently assigned to the requesting machine and embeds them directly inside the JWT's payload as a permissions claim. Because the token is cryptographically signed, a receiving service can trust the embedded permissions without needing to call back to ZEB for every request.

Each protected API defines which permission string it requires for a given action. When a request arrives, the service: (1) validates the JWT signature and expiry, (2) reads the permissions claim from the decoded token, and (3) checks whether the required permission string is present. If it is, the request proceeds; if not, the service returns HTTP 403 Forbidden and the decision is written to the audit log.

## 12. Database Design

---

### 12.1 Machines

Stores every registered machine identity and its current status.

| Column             | Type                   | Description                                     |
|--------------------|------------------------|-------------------------------------------------|
| machine_id         | UUID (PK)              | Unique identifier for the machine               |
| name               | VARCHAR                | Human-readable service name, e.g. order-service |
| owner              | VARCHAR                | Team or person responsible for the machine      |
| client_id          | VARCHAR (unique)       | Public identifier used during authentication    |
| client_secret_hash | VARCHAR                | Salted hash of the client secret                |
| status             | ENUM(active, disabled) | Whether the machine may currently authenticate  |
| created_at         | TIMESTAMP              | Registration time                               |

### 12.2 Permissions

Stores the permission strings assigned to each machine.

| Column        | Type                  | Description                                 |
|---------------|-----------------------|---------------------------------------------|
| permission_id | UUID (PK)             | Unique identifier for the permission record |
| machine_id    | UUID (FK -> Machines) | The machine this permission belongs to      |
| permission    | VARCHAR               | Permission string, e.g. inventory:read      |
| created_at    | TIMESTAMP             | When the permission was assigned            |

### 12.3 AuditLogs

Stores an immutable record of authentication and authorization events.

| Column     | Type                  | Description                                                     |
|------------|-----------------------|-----------------------------------------------------------------|
| log_id     | UUID (PK)             | Unique identifier for the log entry                             |
| machine_id | UUID (FK -> Machines) | Machine associated with the event                               |
| event_type | VARCHAR               | e.g. AUTH_SUCCESS, AUTH_FAILURE, TOKEN_VALIDATED, ACCESS_DENIED |
| details    | TEXT                  | Additional context, e.g. which permission was checked           |
| timestamp  | TIMESTAMP             | When the event occurred                                         |

Relationships: one Machine has many Permissions and many AuditLogs (one-to-many in both cases). Separating these into three tables keeps the identity record (Machines), the access rules (Permissions), and the historical record (AuditLogs) independently queryable and easy to extend later.

## 13. REST API Design

---

| Method & Path                   | Description                                                                  |
|---------------------------------|------------------------------------------------------------------------------|
| POST /machines                  | Register a new machine identity.                                             |
| GET /machines                   | List all registered machines, optionally filtered by status or owner.        |
| PATCH /machines/{id}/disable    | Disable a machine, immediately revoking future authentication.               |
| POST /machines/{id}/credentials | Generate a new client_id/client_secret pair for a machine.                   |
| POST /oauth/token               | Authenticate a machine and issue a signed JWT.                               |
| POST /oauth/verify              | Validate a JWT's signature and expiry, returning its decoded claims.         |
| POST /machines/{id}/permissions | Assign one or more permission strings to a machine.                          |
| GET /audit                      | Retrieve audit log entries, optionally filtered by machine_id or event_type. |

## 14. Demo Applications

---

### Scenario 1: Order Service -> Inventory Service

The Order Service authenticates with ZEB, receives a JWT scoped with inventory:read, and calls the Inventory Service to check stock. The Inventory Service validates the token and permission before responding, demonstrating the full authentication and authorization loop.

### Scenario 2: Inventory Service -> Payment Service

After confirming stock, the Inventory Service (acting on behalf of the order flow) calls the Payment Service, which requires a different permission (payment:create). This scenario demonstrates that permissions are enforced independently at each hop, not just at the network edge.

### Scenario 3: Notification Service

Once payment succeeds, the Payment Service calls the Notification Service using its own machine identity and a notification:send permission, showing how a chain of independently-authenticated services can cooperate without ever sharing credentials directly with one another.

## 15. Real-World Use Cases

---

The following examples illustrate, at a conceptual level, how organizations rely on machine identity patterns similar to ZEB's design. Specific internal architectures of these companies are not publicly documented in detail; the descriptions below reflect commonly understood, general patterns in large-scale distributed systems.

### 15.1 E-commerce (Amazon-style)

Order, Inventory, Payment, and Shipping services each need to trust one another without human involvement. A machine identity layer ensures the Payment service only accepts requests from services explicitly permitted to initiate charges.

## 15.2 Streaming (Netflix-style)

A recommendation service, a billing service, and a video-encoding pipeline all call each other constantly. Machine identity prevents an encoding job, for example, from being able to reach billing systems it has no reason to access.

## 15.3 Ride-hailing (Uber-style)

A trip-matching service, a pricing service, and a driver-location service must exchange data in near real time. Verified machine identity keeps location and pricing data from being requested by unrelated internal systems.

## 15.4 Banking

Core banking systems, fraud-detection engines, and notification systems are tightly regulated; machine identities with fine-grained, auditable permissions are essential for compliance and for proving exactly which system performed which action.

## 15.5 Healthcare

Systems handling patient records must restrict which internal services can read or write sensitive data. Machine identity plus audit logging supports the accountability required by healthcare data-protection regulations.

## 15.6 Airlines

Booking, pricing, and check-in systems interconnect across many legacy and modern components; consistent machine identity allows new services to integrate securely without inheriting broad, implicit trust.

## 15.7 Cloud-Native, Microservices, and Kubernetes

In containerized environments, workloads are created and destroyed continuously. Machine identity issued at startup — rather than baked into a container image — is central to a Zero Trust posture where no internal call is trusted by default.

# 16. Testing Strategy

---

Testing will cover the following areas:

- Functional Testing — verifying each endpoint behaves as specified.
- Authentication Testing — valid and invalid credential combinations.
- Authorization Testing — permission checks for allowed and disallowed actions.
- Security Testing — expired tokens, tampered signatures, disabled machines.
- Negative Testing — malformed requests, missing fields, wrong data types.
- API Testing — status codes and response formats across all endpoints.
- JWT Validation Testing — signature verification and expiry enforcement.
- Integration Testing — full demo scenarios across multiple services.

| Test ID | Scenario                            | Input                        | Expected Output                  | Result |
|---------|-------------------------------------|------------------------------|----------------------------------|--------|
| TC-01   | Register a new machine              | Valid name, owner            | 201 Created, machine_id returned | Pass   |
| TC-02   | Authenticate with valid credentials | Correct client_id/secret     | 200 OK, JWT returned             | Pass   |
| TC-03   | Authenticate with invalid secret    | Wrong client_secret          | 401 Unauthorized                 | Pass   |
| TC-04   | Authenticate a disabled machine     | Valid creds, status=disabled | 401 Unauthorized                 | Pass   |
| TC-05   | Validate an expired token           | Token past exp claim         | 401 Unauthorized                 | Pass   |
| TC-06   | Validate a tampered token           | Modified signature           | 401 Unauthorized                 | Pass   |
| TC-07   | Access with missing permission      | Valid token, wrong scope     | 403 Forbidden                    | Pass   |
| TC-08   | Access with correct permission      | Valid token, correct scope   | 200 OK                           | Pass   |
| TC-09   | Retrieve audit logs for a machine   | machine_id filter            | 200 OK, event list               | Pass   |
| TC-10   | List machines by status             | status=active                | 200 OK, filtered list            | Pass   |

## 17. Version Roadmap

---

### Version 1 — Current MVP

Core machine registration, authentication, JWT issuance/validation, basic permission assignment, and audit logging, as described in Section 6.1.

### Version 2 — Possible Enhancements

Adds RBAC-style role grouping, secret rotation, a basic admin dashboard, and improved reporting over audit data — building on the stable V1 core without changing its fundamentals.

### Version 3 — Enterprise Features

Introduces certificate-based authentication and mTLS, multi-tenancy for isolating multiple teams/organizations, and Kubernetes-native integration for issuing identities to pods automatically.

### Version 4 — AI Agent Identity

Extends the identity model to autonomous AI agents that call internal systems on their own, including tighter-scoped, shorter-lived tokens and behavior-aware risk scoring suited to non-deterministic agent actions.

## 18. Future Enhancements

---

- Secret Rotation — automatically issuing new client secrets on a schedule, with a grace period for the old one.
- Certificate-based Authentication — using X.509 certificates instead of shared secrets for stronger identity proof.
- mTLS — mutual TLS so both client and server verify each other's certificate at the network layer.
- SPIFFE — adopting the SPIFFE standard for portable, verifiable workload identities across platforms.
- Kubernetes Integration — automatically issuing identities to pods at startup via a sidecar or admission controller.
- API Gateway Integration — enforcing authentication and authorization centrally at the gateway layer.
- Identity Dashboard — a web UI for managing machines, permissions, and viewing audit history.
- Machine Health Score — a computed indicator of a machine's security posture based on its behavior over time.
- Risk Assessment — flagging unusual authentication patterns for review.
- Zero Trust Policies — moving from static permissions toward continuously evaluated, context-aware access decisions.
- AI Agent Identity — a dedicated identity type and lifecycle for autonomous AI agents, as introduced in Version 4.

## 19. Expected Learning Outcomes

---

By completing this project, the team will gain practical, hands-on understanding of:

- Core IAM concepts: authentication, authorization, and audit, and how they fit together.
- How OAuth2-style client-credentials flows work under the hood.
- How JSON Web Tokens are structured, signed, and verified.
- How to design a permission model and enforce it at the API layer.
- Secure secret handling: hashing, one-time reveal, and why plaintext secrets should never be stored.
- How to design a relational schema for an identity system.
- How to design and document a REST API professionally.
- How to build and reason about multi-service demo systems that depend on trust between components.
- How to write structured functional and security test cases.
- How real-world platforms like Vault, Keycloak, and SPIFFE/SPIRE approach these same problems, giving useful context for future work with those tools.

## 20. Conclusion

---

ZEB gives the team a concrete, working answer to a question every modern engineer eventually faces: how do machines trust each other securely? By building the registration, authentication, token issuance, authorization, and audit pieces from first principles, the team moves beyond treating IAM as a black box and instead understands the mechanics that underpin production systems like Vault, Keycloak, and SPIFFE/SPIRE.

Academically, the project touches cryptography, distributed systems, API design, and database design in a single, cohesive build. Professionally, it produces a portfolio-ready system and a set of skills — secure credential handling, token-based authentication, and permission enforcement — that are directly transferable to real engineering roles. Scoping Version 1 tightly, while documenting a clear path toward certificates, Kubernetes integration, and AI agent identity in later versions, keeps the project achievable now while leaving room to grow.